

TRADE-OFFS

When To (Not) Use DI

IT'S NOT FREE — RECOGNIZE WHEN IT EARNS ITS KEEP

REACH FOR DI WHEN...

- ✓ You have multiple interchangeable implementations — different payment gateways, storage backends, or environments (dev/test/prod).
- ✓ You need to unit test business logic in isolation from external systems (databases, APIs, the file system).
- ✓ The app is large enough that wiring dependencies by hand everywhere becomes repetitive and error-prone.
- ✓ You're following a clean/hexagonal architecture and want business logic decoupled from infrastructure.

THINK TWICE WHEN...

- ✗ The class has one, permanent, simple dependency that will never change or need mocking (e.g. a Math utility).
- ✗ It's a small script or prototype — a full DI framework adds setup overhead with no real payoff yet.
- ✗ The extra indirection makes a small codebase harder to read and trace than it needs to be.
- ✗ Performance is extremely critical and a reflection-based DI framework adds unacceptable overhead (rare, but real).